# DFS Templates Cheat Sheet — The 20% That Covers 80%

---

## Template 1: The Universal Skeleton

Every tree DFS problem is a variation of this:

```python
def dfs(node):
    # 1. Base case
    if node is None:
        return ___  # depends on problem

    # 2. Recurse
    left = dfs(node.left)
    right = dfs(node.right)

    # 3. Combine
    return ___  # this is where the thinking happens
```

---

## Template 2: Compute a Value (height, sum, min, max, count)

```python
def dfs(node):
    if node is None:
        return BASE

    left = dfs(node.left)
    right = dfs(node.right)

    return COMBINE(left, right)
```

### Fill-in Cheat Sheet

| Problem | BASE | COMBINE |
|---|---|---|
| Max depth | 0 | max(left, right) + 1 |
| Sum of values | 0 | node.val + left + right |
| Count nodes | 0 | left + right + 1 |

| Problem | BASE | COMBINE |
|---|---|---|
| Min value | `float('inf')` | `min(node.val, left, right)` |
| Max value | `float('-inf')` | `max(node.val, left, right)` |

---

## Template 3: True/False Check (exists, identical, symmetric)

```python
def dfs(node):
    if node is None:
        return True or False  # True if "empty is valid", False if "need to find something"

    # Check failure conditions first
    if SOMETHING_WRONG:
        return False

    left = dfs(node.left)
    right = dfs(node.right)

    return left OP right  # 'and' = everything must pass, 'or' = any one passes
```

**Quick rule:** "does X exist anywhere" → use `or`. "Is every node valid" → use `and`.

---

## Template 4: Pass Info Downward (path sum, target search)

```python
def dfs(node, state):
    if node is None:
        return False

    if LEAF and CONDITION_MET:
        return True

    left = dfs(node.left, UPDATED_STATE)
    right = dfs(node.right, UPDATED_STATE)

    return left or right
```

**Rule:** When the answer depends on the path from root to current node, pass that info as a parameter.

---

### Template 5: Track a Global Answer (diameter, max path sum)

```python
def solve(root):
    best = 0

    def dfs(node):
        nonlocal best
        if node is None:
            return 0

        left = dfs(node.left)
        right = dfs(node.right)

        best = max(best, SOMETHING(left, right))  # track global answer
        return LOCAL_ANSWER  # return what parent needs

    dfs(root)
    return best
```

**Rule:** When the function needs to return one thing to the parent but track a different thing overall, use `nonlocal`.

---

### Template 6: Two-Tree Comparison (identical, symmetric)

```python
def dfs(node1, node2):
    if node1 is None and node2 is None:
        return True
    if node1 is None or node2 is None:
        return False
    if node1.val != node2.val:
        return False

    return dfs(node1.left, node2.left) and dfs(node1.right, node2.right)
```

For **symmetric** tree: compare `node1.left` with `node2.right` and `node1.right` with `node2.left`.

---

### The Decision Flowchart

When you see a tree problem, ask:

1. **"Am I computing a single value?"** → Template 2

2. **"Am I checking if something is true/false?"** → Template 3
3. **"Do I need info from the root as I go down?"** → Template 4
4. **"Do I need to return X but track Y?"** → Template 5
5. **"Am I comparing two trees?"** → Template 6

---

## Base Case Quick Reference

| **Problem type** | **`node is None` returns** |
|---|---|
| Height / Count | `0` |
| Sum | `0` |
| Find minimum | `float('inf')` |
| Find maximum | `float('-inf')` |
| Exists / Search | `False` |
| All valid | `True` |

---

## Three Traversal Orders

| **Order** | **Process node** | **Pattern** | **Use when** |
|---|---|---|---|
| Preorder | Before calls | `process → left → right` | Need to act before going deeper |
| Inorder | Between calls | `left → process → right` | BST sorted traversal |
| Postorder | After calls | `left → right → process` | Need children's answers first (most common) |